SQL Basic Foundation

1. **Relational Database:** Data সাধারণত **table (rows + columns)** আকারে structured থাকে। **Example:** Customers **tables** id | name | city ---1 | Rahim| Dhaka
2. **Relational Database-এর Predefined Schema** Table তৈরি করার সময় আগে থেকেই **column ও data type** define করতে হয়।
   CREATE TABLE customers (
   id INTEGER,
   name VARCHAR (100),
   age INTEGER
   ); **সহজভাবে:** আগে structure → পরে data।
3. **Non-Relational Database:** Non-relational database তুলনামূলকভাবে **flexible data modeling** allows করে। **সহজভাবে:** Fixed table structure বাধ্যতামূলক নয়।

   বইয়ের ডেটা: { "product_id": 101,
   "name": "হুমায়ূন আহমেদের উপন্যাস সমগ্র",
   "author": "হুমায়ূন আহমেদ",
   "pages": 450}

   স্মার্টফোনের ডেটা: { "product_id": 102,
   "name": "Galaxy S26",
   "brand": "Samsung",
   "ram": "12GB",
   "battery": "5000mAh"}

দুটি প্রোডাক্ট একই **Database Connection** (টেবিলের মতো অংশ) থাকলেও তাদের ভেতরের তথ্যের ধরন ও ফিল্ড সম্পূর্ণ আলাদা। SQL ডেটাবেস হলে আপনাকে ফাঁকা ফিল্ডগুলোর জন্য NULL ভ্যালু রাখতে হতো, কিন্তু NoSQL-এ তার প্রয়োজন নেই।

4. **Common Relational Databases:**
   - **MySQL | PostgreSQL | SQLite | SQL Server**

   Node.js/Node database নয়—এটি JavaScript runtime।
   (MySQL/PostgreSQL): এটি হলো রেস্তোরাঁর **রান্নাঘর বা স্টোররুম**, যেখানে সব খাবার (ডেটা) সাজানো থাকে।
   Node.js: এটি হলো রেস্তোরাঁর **ওয়েটার**। সে কাস্টমারের কাছ থেকে অর্ডার নিয়ে রান্নাঘরে (ডেটাবেসে) যায়, খাবার (ডেটা) নিয়ে এসে আবার কাস্টমারকে দেয়। ওয়েটার নিজে খাবার তৈরি বা জমা রাখে না।
5. **PostgreSQL:** PostgreSQL-এর শক্তিশালী **SQL support এবং advanced features** রয়েছে।
   মনে রাখুন: PostgreSQL = Powerful relational database — ব্যাকএন্ড ডেভলপমেন্টের দুনিয়ায় এটিকে "ডেটাবেসের সুইজারল্যান্ড আর্মি নাইফ" বলা হয়।

কেন PostgreSQL এত শক্তিশালী?

Advanced Data Types: এটি শুধু সাধারণ টেক্সট বা নাম্বার নয়, বরং **JSON/JSONB**, জ্যামিতিক ডেটা (Coordinates), এবং নেটওয়ার্ক অ্যাড্রেস সরাসরি স্টোর করতে পারে। JSONB সাপোর্ট থাকার কারণে এটি রিলেশনাল হয়েও (NoSQL) এর মতো কাজ করতে পারে।

Extensibility: এই ডেটাবেসের ভেতরে নিজস্ব ফাংশন, কাস্টম ডেটা টাইপ, এমনকি অন্য ল্যাঙ্গুয়েজ (যেমন Python বা Tcl) দিয়ে কোড লিখতে পারেন।

Concurrency (MVCC): একই সময়ে হাজার হাজার মানুষ ডেটা রিড এবং রাইট করলেও এটি ডেটা লক না করে চমৎকারভাবে হ্যান্ডেল করতে পারে (Multi-Version Concurrency Control)।

Enterprise Features: এতে আছে ফুল-টেক্সট সার্চ (Elasticsearch-এর মতো বেসিক সার্চিং বিল্ট-ইন পাওয়া যায়) এবং ভৌগোলিক ডেটা অ্যানালাইসিসের জন্য বিখ্যাত **Postages** এক্সটেনশন।

6. **SQLite:** ডেভলপারদের ভাষায় এটিকে বলা হয় **"Zero-Configuration"** বা কোনো ঝামেলা ছাড়া ডেটাবেস।

বৈশিষ্ট্যের

File-based: সাধারণ ডেটাবেসের মতো এর জন্য কোনো বিশাল সফটওয়্যার ব্যাকগ্রাউন্ডে ইনস্টল করতে হয় না। আপনার প্রজেক্ট ফোল্ডারের ভেতরে জাস্ট একটি সাধারণ ফাইল (যেমন: database. SQLite) হিসেবে এটি তৈরি হয়। আপনার সব ডেটা ওই একটা ফাইলের ভেতরেই থাকে।

No Server Needed: MySQL বা PostgreSQL চালাতে হলে কম্পিউটারে আলাদা একটি লোকাল সার্ভার ব্যাকগ্রাউন্ডে রান করে রাখতে হয় এবং নির্দিষ্ট পোর্ট (যেমন: 3306 বা 5432) ব্যবহার করতে হয়। SQLite-এর কোনো সার্ভার নেই, এটি সরাসরি আপনার অ্যাপ্লিকেশনের কোডের সাথে জুড়ে (Embedded) থাকে।

Lightweight: এর সাইজ মাত্র কয়েক মেগাবাইট। মেমোরি বা র‍্যাম খরচ করে না বললেই চলে, কিন্তু কাজের বেলায় এটি পুরো SQL ল্যাঙ্গুয়েজ সাপোর্ট করে।

Open-source + Relational: এটি সম্পূর্ণ ফ্রি এবং ওপেন-সোর্স। হালকা হলেও এটি টেবিল, রো, ফরেন-কি (Foreign Key) এবং রিলেশন পুরোপুরি মেইনটেইন করতে পারে।

(Real-world Analogy)

PostgreSQL/MySQL: হলো একটি কোল্ড স্টোরেজ: বিশাল গোডাউন, আলাদা সিকিউরিটি গার্ড (সার্ভার) লাগে, মেইনটেইন করার খরচ ও নিয়মকানুন বেশি।

SQLite: হলো আপনার পকেটের টিফিন বক্স: কমপ্যাক্ট, আপনার সাথেই ঘোরে, আলাদা কোনো গার্ড লাগে না, ঝটপট খুলে খাবার (ডেটা) খাওয়া যায়।

7. **INTEGER:** Whole numbers রাখার জন্য INTEGER ব্যবহার করা হয়। age INTEGER CHECK (age >= 18 AND age <= 25).
8. **TEXT:** Long text বা বড় string রাখার জন্য TEXT ব্যবহার করা যায়। description TEXT
   Example: "This is a very long product desc.."
9. **BOOLEAN:** শুধু **TRUE/FALSE** value রাখে।
10. **SELECT *:** এখানে * মানে **সব columns**।
    Example: SELECT * FROM customers;
    → customers table-এর সব column দেখাবে।
11. **INSERT INTO:** নতুন record/row যোগ করতে ব্যবহার করা হয়। INSERT INTO products (name, price) VALUES ('Laptop', 50000);
12. **WHERE:** Data **filter** করার জন্য WHERE ব্যবহার করা হয়। SELECT price FROM products WHERE category = 'Laptops';
    মানে শুধু **Laptops category-এর products** আসবে।

(Real-world Analogy)

মনে করুন, আপনি একটি লাইব্রেরিতে গিয়ে লাইব্রেরিয়ানকে বললেন, "আমাকে বই দিন" (SELECT * FROM library;)। তিনি আপনাকে সব বই এনে দেবেন। কিন্তু আপনি যদি বলেন, "আমাকে শুধু **হুমায়ূন আহমেদের** বই দিন" (SELECT * FROM library WHERE author = 'Humayun Ahmed';), তাহলে তিনি ফিল্টার করে শুধু ওই লেখকের বইগুলোই আপনার সামনে রাখবেন। এই ফিল্টার করার কাজটিই করে WHERE।

13. **Primary Key:** Primary Key প্রতিটি record-কে **uniquely identify** করে। id---1 | 2 | 3 id INTEGER PRIMARY KEY
    মনে রাখুন: Primary Key = Unique Identity
    "মানুষের যেমন একটি ইউনিক **(NID)** থাকে, ডেটাবেসের টেবিলের প্রতিটি **row** জন্য তেমনি থাকে একটি **Primary Key**।"
14. **Foreign Key:** Foreign Key অন্য table-এর সাধারণত **Primary Key-কে reference** করে। Example: **Customers:** customer_id->1,2 **Orders:** order_id->101, 102 | customer_id 1, 2 এখানে Orders.customer_id হলো **Foreign Key**।

একটি টেবিলের ভেতরের রাজা হলো তার **Primary Key**। কিন্তু সেই রাজা যখন অন্য কোনো টেবিলে বেড়াতে যায়, তখন সে হয়ে যায় **Foreign Key**।

15. **Primary Key-এর Characteristics:** Primary Key: Duplicate হতে পারে না | NULL হতে পারে না | Unique হতে হবে

মনে রাখুন: PK = Unique + Not NULL

একটি টেবিলে একাধিক কলামকে ইউনিক (Unique) করা যেতে পারে (যেমন ইমেইল, ফোন নম্বর), কিন্তু **প্রাইমারি কি থাকবে কেবল একটিই**। এটি ওই টেবিলের প্রধান পরিচয়পত্র।

16. **Composite Primary Key:** এমন প্রাইমারি কি, যা একটি মাত্র কলামের পরিবর্তে দুই বা ততোধিক কলামের সংমিশ্রণে (Combination) তৈরি হয়.

ধরে নিন, একটি বিশ্ববিদ্যালয়ের ডেটাবেসে ছাত্র-ছাত্রীদের কোর্স রেজিস্ট্রেশনের তথ্য রাখা হচ্ছে।

- একজন ছাত্র একাধিক কোর্স নিতে পারে।
- একটি কোর্সে একাধিক ছাত্র থাকতে পারে।

student_id বা শুধু course_id-কে প্রাইমারি কি বানাই, তবে তা ডুপ্লিকেট হয়ে যাবে. তাই আমরা student_id এবং course_id এই দুটি কলামকে একসাথে মিলিয়ে একটি Composite Primary Key তৈরি করব.

17. **One-to-Many Relationship:** একটি record → অন্য table-এর **অনেক records**।

One Customer → Many Orders

One User → **Many** Blog Posts (একজন ইউজার অনেকগুলো ব্লগ পোস্ট লিখতে পারেন)।

One Department → **Many** Employees (একটি বিভাগে অনেকজন কর্মী কাজ করতে পারেন)।

One YouTube Channel → **Many** Videos (একটি চ্যানেলে অনেকগুলো ভিডিও থাকতে পারে)।

18. **Many-to-Many Relationship:** এক table-এর অনেক records ↔ অন্য table-এর অনেক records।
Students ↔ Courses

সাধারণত এখানে **junction/bridge table** ব্যবহার করা হয়।

Many Authors ↔ **Many** Books (একটি বই একাধিক লেখক লিখতে পারেন, আবার একজন লেখক একাধিক বইও লিখতে পারেন)।

Many Users ↔ **Many** Liked Videos (একজন ইউজার অনেক ভিডিও লাইক করতে পারেন, আবার একটি ভিডিও অনেক ইউজার লাইক করতে পারেন)।

Many-to-Many হলো এমন এক সম্পর্ক যা সরাসরি জোড়া লাগে না। এদের মাঝে মিলন ঘটাতে সবসময় একটি তৃতীয় টেবিল বা **Junction/Bridge Table**-এর সাহায্য নিতে হয়।

19. **Self-Referencing Relationship:** একটি table-এর record যখন **একই table-এর অন্য record-কে reference করে**।

Employees & Managers

একটি কোম্পানির সবাই আসলে কর্মচারী (Employee), এমনকি ম্যানেজার বা সিইও (CEO)-ও একজন কর্মচারী। সবার তথ্যই একটি সিঙ্গেল employees টেবিলে থাকে। কিন্তু কে কার অধীনে কাজ করছে (কার ম্যানেজার কে) তা বোঝাতে এই রিলেশন ব্যবহার করা হয়।

Multi-level Categories

একটি ই-কমার্স সাইটে মেইন ক্যাটাগরি এবং সাব-ক্যাটাগরি থাকে (যেমন: Electronics -> Laptops -> Gaming Laptops)। এগুলো আলাদা টেবিল না বানিয়ে একটি মাত্র টেবিলেই রাখা যায়।

20. **INNER JOIN:** শুধু দুই table-এর **matching records** দেয়। **সহজভাবে:** Table A ∩ Table B শুধু match।

Admin Dashboard-: "কোন কাস্টমার কোন খাবারটি অর্ডার করেছেন এবং তার দাম কত?"
আপনি যদি শুধু orders টেবিল দেখেন, তবে কাস্টমারের নাম পাবেন না (শুধু আইডি পাবেন)। আবার শুধু customers টেবিল দেখলে খাবারের নাম পাবেন? না। তাই আপনাকে দুটি টেবিল জোড়া লাগাতে হবে।

21. **LEFT JOIN:** Left table-এর **সব records** রাখে।

Table A (Left) এর সব ডেটা আসবে + Table B (Right) এর শুধু ম্যাচিং ডেটা আসবে। যদি ডান পাশের টেবিলে কোনো মিল না পাওয়া যায়, তবে সেই ঘরগুলো **NULL** (ফাঁকা) দেখাবে।

আপনার কোম্পানির মার্কেটিং টিম বলল, "আমাদের অ্যাপের সব কাস্টমারের তালিকা দিন এবং দেখুন কে কী অর্ডার করেছে। যারা এখনো কোনো অর্ডার করেনি, তাদের নামের পাশে ফাঁকা থাকুক (যাতে আমরা তাদের ডিসকাউন্ট কুপন পাঠাতে পারি)।"

22. **psql কমান্ডের শর্ট-লিস্ট**

\l ➔ Lists all databases

\d ➔ Lists all tables]

\dn ➔ Lists all schemas

\c ➔ Connects to a database

23. **UNIQUE Constraint**-এর মূল কাজ হলো ওই কলামে কোনো ডুপ্লিকেট ডেটা ঢুকতে দেবে না।
টুইস্ট এখানে: এটি কিন্তু **NULL ভ্যালু এলাউ করে**।

24. **FULL JOIN:** দুই table-এর **সব records** রাখে।

Match থাকলে data combine করবে।Match না থাকলে অন্য পাশের column-এ NULL থাকবে।

Note: **MySQL সরাসরি FULL JOIN support করে না**; PostgreSQL-এ FULL JOIN available।

FULL JOIN (বা FULL OUTER JOIN) দুই টেবিলের কাউকেই ফেলে দেয় না। মিল থাকলে তো ডেটা জোড়া লাগাবেই, আর মিল না থাকলে দুই পাশের টেবিলের ডেটাই আলাদাভাবে দেখাবে এবং যে পাশে মিল নেই সে পাশে NULL বসিয়ে দেবে।

ধরে নিন, আপনার ইউনিভার্সিটিতে একটা কনসার্ট হচ্ছে। আপনার কাছে দুটি টেবিল আছে:

registrations (Left Table): কারা কনসার্টের টিকিট কেটেছে। Sakib (ID 1) | Maria (ID 2)

tshirts (Right Table): কারা অলরেডি টি-শার্টের সাইজ বুকিং দিয়ে রেখেছে। Maria (ID 2 - Size M) | John (ID 3 - Size XL)

LEFT JOIN-এর খেলা (Ticket Centric): ম্যানেজমেন্ট বলল: *"যারা টিকিট কেটেছে (Left Table) তাদের লিস্ট দাও, আর পাশে দেখাও কে কোন সাইজের টি-শার্ট পাবে।"*
SQL
SELECT registrations.name, tshirts. Size
FROM **registrations**
LEFT JOIN **tshirts** ON registrations.id = tshirts.id;
আউটপুট (Result):

name	size
Sakib	**NULL** *(টিকিট কেটেছে কিন্তু টি-শার্টের সাইজ দেয় নাই)*
Maria	M *(টিকিটও কেটেছে, সাইজও দিয়েছে)*

কী ঘটল? John কিন্তু আউটপুট থেকে সম্পূর্ণ গায়েব! কারণ জন টিকিট কাটে নাই (সে Left টেবিলে ছিল না)। LEFT JOIN ডান টেবিলের এই অতিরিক্ত ডেটাকে ডিলিট করে দেয়।

FULL JOIN-এর খেলা (Audit / Error Finding Mode)

এবার একাউন্টস ডিপার্টমেন্ট এসে বলল: *"আমার টিকিট এবং টি-শার্ট দুইটার সব ডেটা একসাথে লাগবে। কেউ যেন বাদ না পড়ে! আমি দেখতে চাই কোথায় কী গরমিল হইছে।"*

SQL

```
SELECT registrations.name, tshirts.size
FROM registrations
FULL JOIN tshirts ON registrations.id = tshirts.id;
```

Use code with caution.

আউটপুট (Result):

name	size
Sakib	**NULL** *(টিকিট কেটেছে কিন্তু টি-শার্টের সাইজ দেয় নাই)*
Maria	M *(টিকিটও কেটেছে, সাইজও দিয়েছে)*
NULL	**XL** *(এই যে জন! টিকিট কাটে নাই, কিন্তু টি-শার্টের সাইজ ঠিকই বুক করে রাখছে!)*

কী ঘটল? John-এর ডেটা কিন্তু সেভ হয়ে গেল! তার নামের কলামে NULL এসেছে কারণ সে Left টেবিলে ছিল না, কিন্তু ডান টেবিলের টি-শার্টের সাইজ XL ঠিকই শো করছে। FULL JOIN দুই পাশের কাউকেই বাদ দেয় না।

মূল পার্থক্য এক নজরে (Summary)

LEFT JOIN: শুধু টিকিট হোল্ডারদের রাখবে। টিকিট ছাড়া কেউ টি-শার্ট বুক করলেও তার ডেটা **ডিলিট** করে দেবে।

FULL JOIN: টিকিট হোল্ডারদেরও রাখবে, আবার টিকিট ছাড়া যারা টি-শার্ট বুক করছে তাদেরও রাখবে (যাতে ভুলগুলো ধরা পড়ে) ।